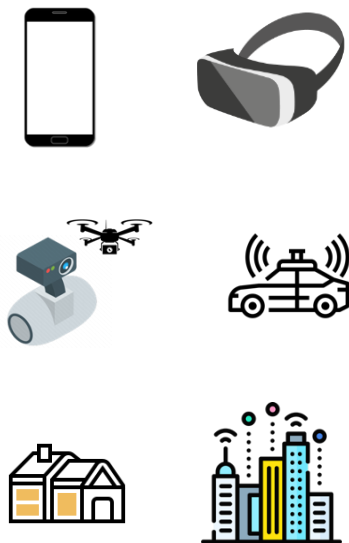


Today's Outline

- Real-time 3D System: reconstruction, rendering, and mapping

The 3D Physical World

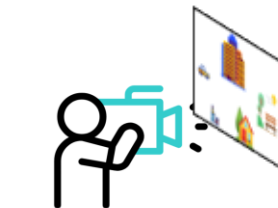


Sensors & Devices

Dynamic Subjects



Dynamic Subjects



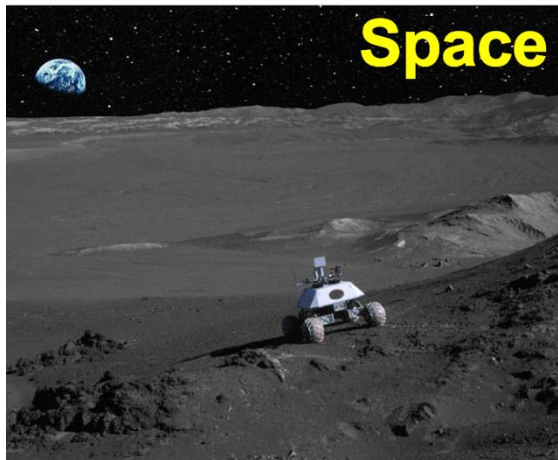
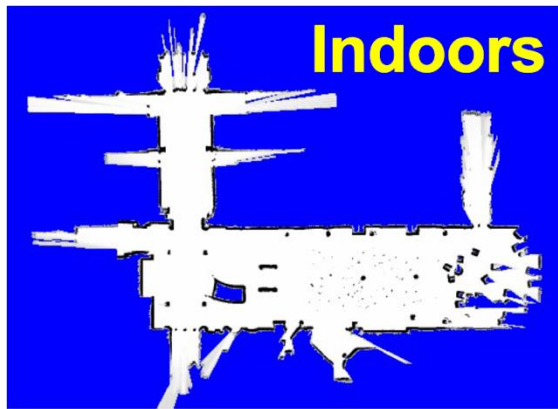
Observer (Camera Pose)



Static 3D Environment



Mapping Applications



Simultaneous Localization and Mapping (SLAM)

Given

- ▣ The robot's controls

$$u_{1:T} = \{u_1, u_2, u_3, \dots, u_T\}$$

- ▣ Observations

$$z_{1:T} = \{z_1, z_2, z_3, \dots, z_T\}$$

Wanted

- ▣ Map of the environment

$$m$$

- ▣ Path of the robot

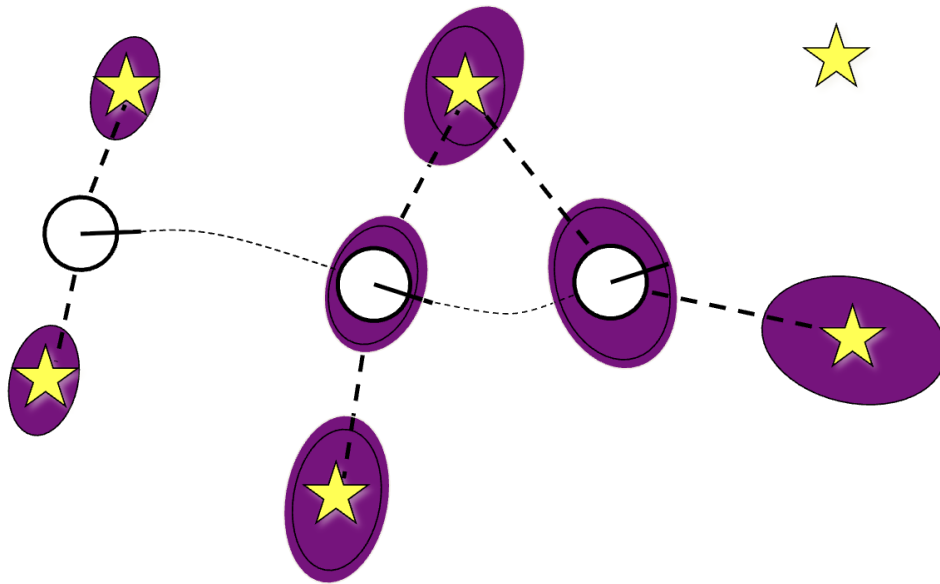
$$x_{0:T} = \{x_0, x_1, x_2, \dots, x_T\}$$

Simultaneous Localization and Mapping (SLAM)



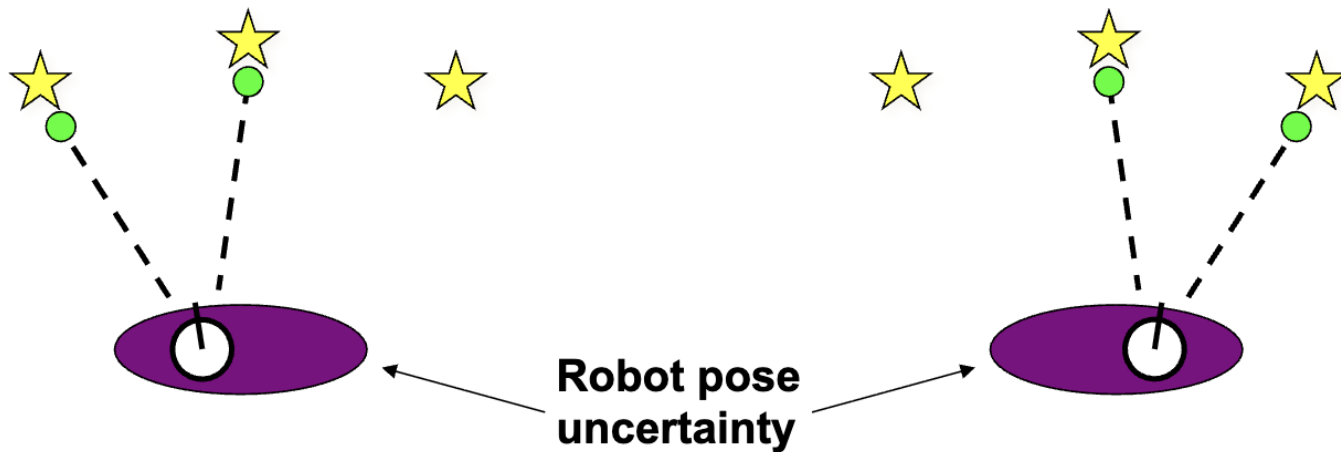
- Building a map and locating the robot in the map at the same time
- Chicken-and-egg problem

Why SLAM is Hard



Robot path error correlates errors in the map

Why SLAM is Hard



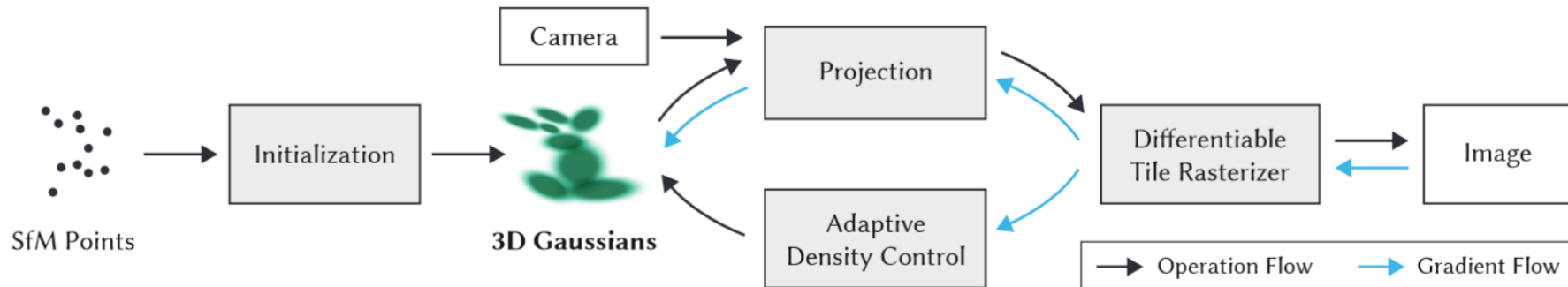
- In the real world, the mapping between observations and landmarks is unknown
- Picking wrong data associations can have catastrophic consequences
- Pose error correlates data associations

SLAM Categories

SLAM Maps can be categorized as either sparse or dense. Achieving real-time dense construction with just single camera (monocular) is challenging. Existing real-time monocular SLAM systems uses mostly sparse methods.

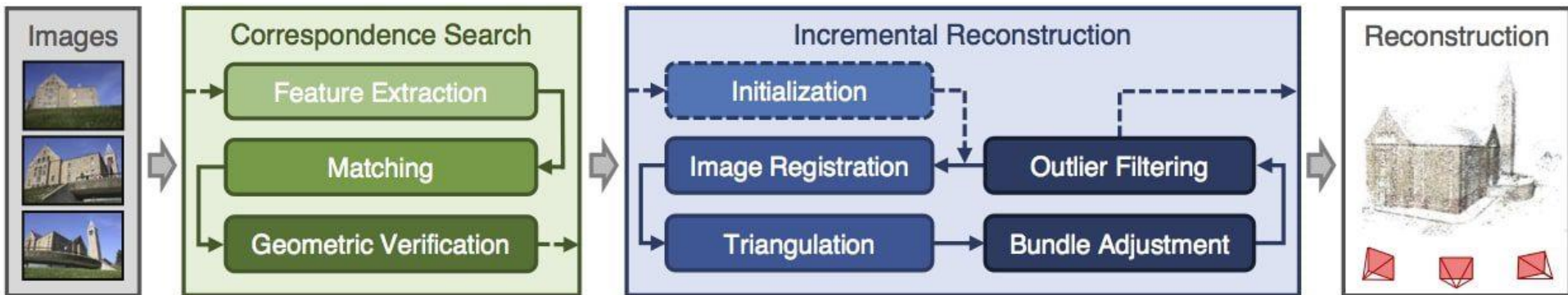
- **Sparse SLAM** methods, such as ORB-SLAM track a handful of discrete, distinctive keypoints. These methods are lightweight, fast and provides accurate camera pose tracking in real time. However, the resulting maps are under modelled (less-detailed). Applications that requires rich scene understanding for immersive AR-VR experience or collision avoidance requires sub-pixel level accurate mapping.
- On the other hand, **Dense SLAM** approaches estimate per-pixel level accurate features from depth across the entire image. This creates a detailed representation of the environment and predicts accurate camera trajectory. However, Dense SLAM is a inverse problem of large dimensionality (computationally intensive) which limits them to operate real time compared to sparse methods. While Droid-SLAM is promising and robust, it still lacks 3D geometric consistency.

Let's Consider the Front End

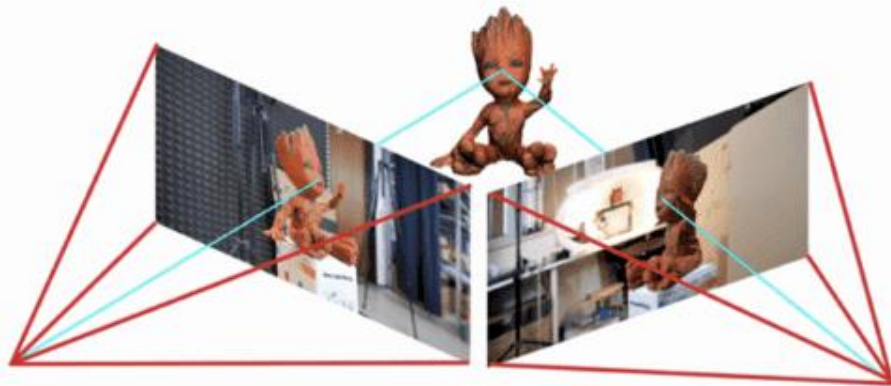
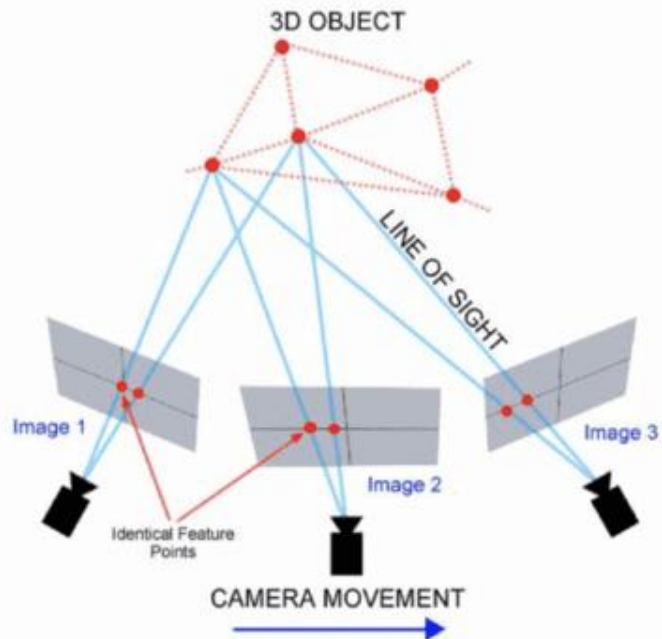


Optimization in the 3D space: slow but accurate

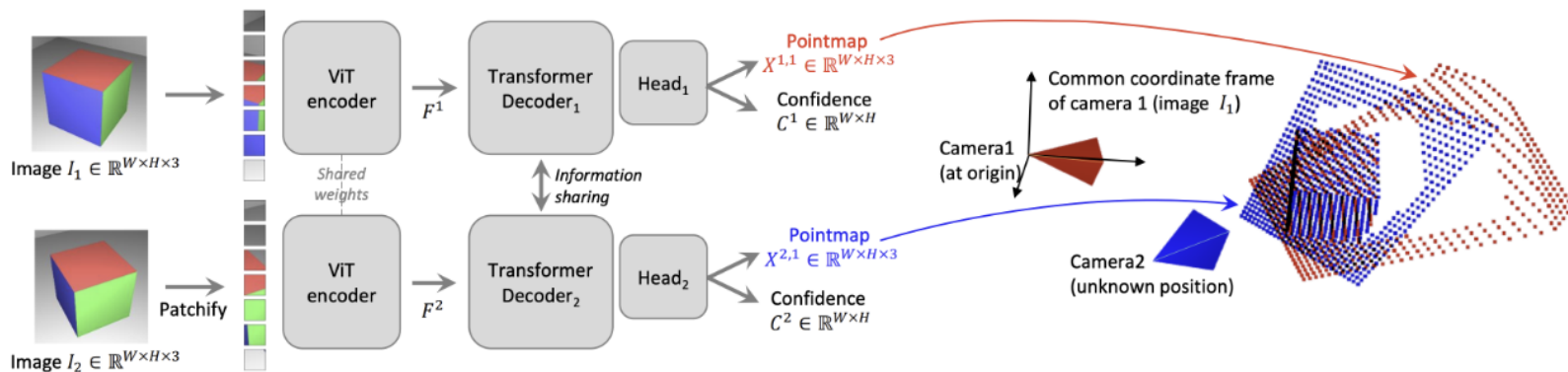
Let's Consider the Front End



Let's Consider the Front End



Let's Consider the Front End



learn a function $F : (I_1, I_2) \rightarrow (P_1, P_2)$

mapping images $I_i[y, x] = (R, G, B)$ to **point maps** $P_i[y, x] = (X, Y, Z)$.

Feed-forward prediction: fast but less accurate

Let's Consider the Front End

4. Experiments with DUS_t3R

Training data. We train our network with a mixture of eight datasets: Habitat [82], MegaDepth [47], ARK-itScenes [22], Static Scenes 3D [56], Blended MVS [122], ScanNet++ [126], CO3D-v2 [75] and Waymo [94]. These datasets feature diverse scene types: indoor, outdoor, landmarks, synthetic, real-world, object-centric, etc. When image pairs are not directly provided with the dataset, we extract them based on the method described in [113]. Specifically, we utilize off-the-shelf image retrieval and point matching algorithms to match and verify image pairs. All in all, we extract 8.5M pairs in total.

Let's Consider the Front End

Methods	Train	Outdoor					Indoor				
		DDAD ^[38]		KITTI ^[34]		BONN ^[64]	NYUD-v2 ^[89]		TUM ^[91]		
		Rel $\downarrow$	$\delta_{1.25}$ $\uparrow$	Rel $\downarrow$	$\delta_{1.25}$ $\uparrow$		Rel $\downarrow$	$\delta_{1.25}$ $\uparrow$	Rel $\downarrow$	$\delta_{1.25}$ $\uparrow$	
DPT-BEiT ^[72]	D	10.70	84.63	9.45	89.27	-	-	5.40	96.54	10.45	89.68
NeWCRFs ^[132]	D	9.59	82.92	5.43	91.54	-	-	6.22	95.58	14.63	82.95
Monodepth2 ^[36]	SS	23.91	75.22	11.42	86.90	56.49	35.18	16.19	74.50	31.20	47.42
SC-SfM-Learners ^[6]	SS	16.92	77.28	11.83	86.61	21.11	71.40	13.79	79.57	22.29	64.30
SC-DepthV3 ^[93]	SS	14.20	81.27	11.79	86.39	12.58	88.92	12.34	84.80	16.28	79.67
MonoViT ^[138]	SS	-	-	09.92	90.01	-	-	-	-	-	-
RobustMIX ^[74]	T	-	-	18.25	76.95	-	-	11.77	90.45	15.65	86.59
SlowTv ^[90]	T	12.63	79.34	(6.84)	(56.17)	-	-	11.59	87.23	15.02	80.86
DUS3R 224-NoCroCo	T	19.63	70.03	20.10	71.21	14.44	86.00	14.51	81.06	22.14	66.26
DUS3R 224	T	16.32	77.58	16.97	77.89	11.05	89.95	10.28	88.92	17.61	75.44
DUS3R 512	T	13.88	81.17	10.74	86.60	8.08	93.56	6.50	94.09	14.17	79.89

Methods	Co3Dv2			RealEstate10K
	RRA@15	RTA@15	mAA(30)	mAA(30)
RelPose ^[135]	57.1	-	-	-
Colmap+SPSG ^[23, 81]	36.1	27.3	25.3	45.2
PixSfM ^[50]	33.7	32.9	30.1	49.4
PosReg ^[104]	53.2	49.1	45.0	-
PoseDiffusion ^[104]	80.5	79.8	66.5	48.0
DUST3R 512 (w/ PnP)	94.3	88.4	77.2	61.2
DUST3R 512 (w/ GA)	96.2	86.8	76.7	67.7

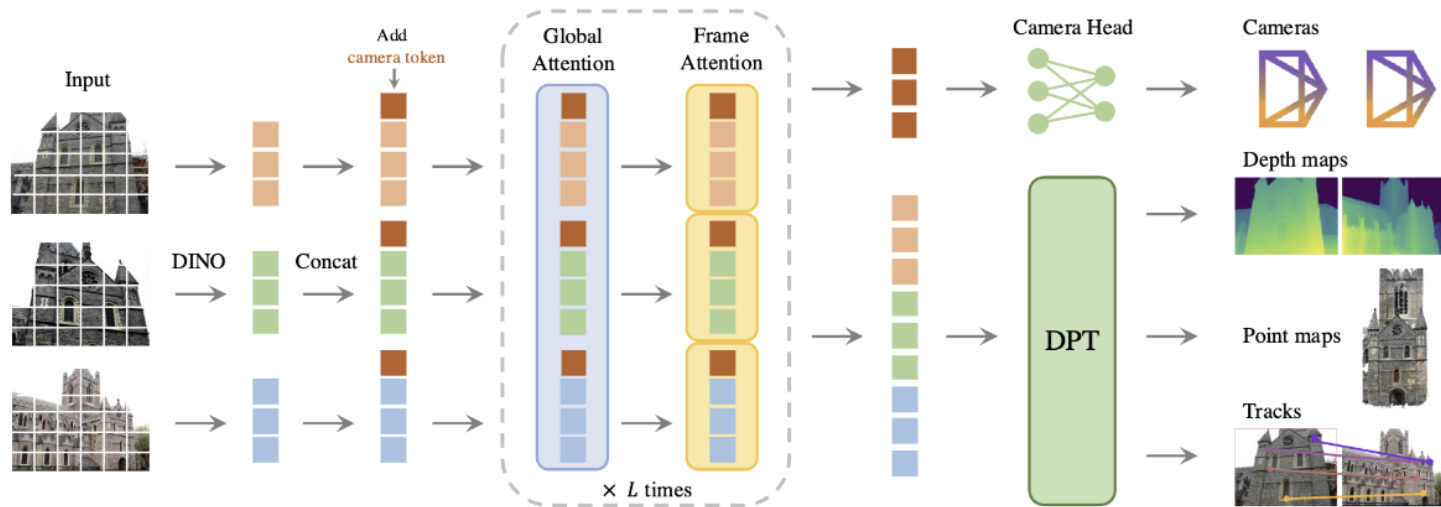
Table 2. **Left:** Monocular depth estimation on multiple benchmarks. D-Supervised, SS-Self-supervised, T-transfer (zero-shot). (Parentheses) refers to training on the same set. **Right:** Multi-view pose regression on the CO3Dv2^[75] and RealEst10K^[140] with 10 random frames.

Methods	GT	GT	GT	Align	KITTI		ScanNet		ETH3D		DTU		T&T		Average	
	Pos	Range	Intrinsics		rel↓	τ ↑	rel↓	τ ↑	rel↓	τ ↑	rel↓	τ ↑	rel↓	τ ↑	rel↓	τ ↑ time (s)↓
(a) COLMAP ^[83, 84]	✓	×	✓	×	12.0	58.2	14.6	34.2	16.4	55.1	0.7	96.5	2.7	95.0	9.3	67.8 $\approx$ 200
COLMAP Dense ^[83, 84]	✓	×	✓	×	26.9	52.7	38.0	22.5	89.8	23.2	20.8	69.3	25.7	76.4	40.2	48.8 $\approx$ 200
MVSNet ^[121]	✓	✓	✓	✓	22.7	36.1	24.6	20.4	35.4	31.4	(1.8)	(86.0)	8.3	73.0	18.6	49.4 0.07
(b) MVSNet Inv. Depth ^[121]	✓	✓	✓	✓	18.6	30.7	22.7	20.9	21.6	35.6	(1.8)	(86.7)	6.5	74.6	14.2	49.7 0.32
Vis-MVSNet ^[134]	✓	✓	✓	×	9.5	55.4	8.9	33.5	10.8	43.3	(1.8)	(87.4)	4.1	87.2	7.0	61.4 0.70
DeMon ^[102]	✓	×	✓	×	16.7	13.4	75.0	0.0	19.0	16.2	23.7	11.5	17.6	18.3	30.4	11.9 0.08
DeepV2D KITTI ^[98]	✓	×	✓	×	(20.4)	(16.3)	25.8	8.1	30.1	9.4	24.6	8.2	38.5	9.6	27.9	10.3 1.43
DeepV2D ScanNet ^[98]	✓	×	✓	×	61.9	5.2	(3.8)	(60.2)	18.7	28.7	9.2	27.4	33.5	38.0	25.4	31.9 2.15
(c) MVSNet ^[121]	✓	×	✓	×	14.0	35.8	1568.0	5.7	507.7	8.3	(4429.1)	(0.1)	118.2	50.7	1327.4	20.1 0.15
MVSNet Inv. Depth ^[121]	✓	×	✓	×	29.6	8.1	65.2	28.5	60.3	5.8	(28.7)	(48.9)	51.4	14.6	47.0	21.2 0.28
Vis-MVSNet ^[134]	✓	×	✓	×	10.3	54.4	84.9	15.6	51.5	17.4	(374.2)	(1.7)	21.1	65.6	108.4	31.0 0.82
Robust MVD Baseline ^[86]	✓	×	✓	×	7.1	41.9	7.4	38.4	9.0	42.6	2.7	82.0	5.0	75.1	6.3	56.0 0.06
DeMon ^[102]	×	×	✓	t	15.5	15.2	12.0	21.0	17.4	15.4	21.8	16.6	13.0	23.2	16.0	18.3 0.08
DeepV2D KITTI ^[98]	×	×	med	(3.1)	(74.9)	23.7	11.1	27.1	10.1	24.8	8.1	34.1	9.1	22.6	22.7	2.07
DeepV2D ScanNet ^[98]	×	×	med	10.0	36.2	(4.4)	(54.8)	11.8	29.3	7.7	33.0	8.9	46.4	8.6	39.9	3.57
(d) DUST3R 224-NoCroCo	×	×	×	med	15.14	21.16	7.54	40.00	9.51	40.07	3.56	62.83	11.12	37.90	9.37	40.39 0.05
DUST3R 224	×	×	×	med	15.39	26.69	(5.86)	(50.84)	4.71	61.74	2.76	77.32	5.54	56.38	6.85	54.59 0.05
DUST3R 512	×	×	med	9.11	39.49	(4.93)	(60.20)	2.91	76.91	3.52	69.33	3.17	76.68	4.73	64.52	0.13

	Methods	GT cams	Acc.↓	Comp.↓	Overall↓
(a)	Camp ^[11]	✓	0.835	0.554	0.695
	Furu ^[32]	✓	0.613	0.941	0.777
	Tola ^[100]	✓	0.342	1.190	0.766
	Gipuma ^[33]	✓	0.283	0.873	0.578
(b)	MVSNet ^[121]	✓	0.396	0.527	0.462
	CVP-MVSNet ^[119]	✓	0.296	0.406	0.351
	UCS-Net ^[16]	✓	0.338	0.349	0.344
	CER-MVS ^[55]	✓	0.359	0.305	0.332
	CIDER ^[118]	✓	0.417	0.437	0.427
	PatchmatchNet ^[103]	✓	0.427	0.277	0.352
	GeoMVSNet ^[136]	✓	0.331	0.259	0.295
	DUST3R 512	×	2.677	0.805	1.741

Table 3. **Left:** Multi-view depth evaluation with different settings: a) Classical approaches; b) with poses and depth range, without alignment; c) absolute scale evaluation with poses, without depth range and alignment; d) without poses and depth range, but with alignment. (Parentheses) denote training on data from the same domain. The best results for each setting are in **bold**. **Right:** MVS results on the DTU dataset, in *mm*. Traditional handcrafted methods (a) have been overcome by learning-based approaches (b) that train on this specific domain.

Let's Consider the Front End



Train a large transformer to take (hundreds of) images in and output lots of 3D representations: camera pose, point maps, tracks.

Feed-forward prediction: fast but less accurate

Let's Consider the Front End

Methods	Re10K (<i>unseen</i>) AUC@30 $\uparrow$	CO3Dv2 AUC@30 $\uparrow$	Time
Colmap+SPSG [92]	45.2	25.3	~ 15 s
PixSfM [66]	49.4	30.1	> 20 s
PoseDiff [124]	48.0	66.5	~ 7 s
DUST3R [129]	67.7	76.7	~ 7 s
MASt3R [62]	76.4	81.8	~ 9 s
VGGSfM v2 [125]	78.9	83.4	~ 10 s
MV-DUST3R [111] †	71.3	69.5	~ 0.6 s
CUT3R [127] †	75.3	82.8	~ 0.6 s
FLARE [156] †	78.8	83.3	~ 0.5 s
Fast3R [141] †	72.7	82.5	~ 0.2 s
Ours (Feed-Forward)	<u>85.3</u>	<u>88.2</u>	~ 0.2 s
Ours (with BA)	93.5	91.8	~ 1.8 s

Table 1. **Camera Pose Estimation on RealEstate10K [161] and CO3Dv2 [88]** with 10 random frames. All metrics the higher the better. None of the methods were trained on the Re10K dataset. Runtime were measured using one H100 GPU. Methods marked with † represent concurrent work.

Known GT camera	Method	Acc. $\downarrow$	Comp. $\downarrow$	Overall $\downarrow$
$\checkmark$	Gipuma [40]	0.283	0.873	0.578
$\checkmark$	MVSNet [144]	0.396	0.527	0.462
$\checkmark$	CIDER [139]	0.417	0.437	0.427
$\checkmark$	PatchmatchNet [121]	0.427	0.377	0.417
$\checkmark$	MASt3R [62]	0.403	0.344	0.374
$\checkmark$	GeoMVSNet [157]	0.331	0.259	0.295
$\times$	DUST3R [129]	2.677	0.805	1.741
$\times$	Ours	0.389	0.374	0.382

Table 2. **Dense MVS Estimation on the DTU [51] Dataset.** Methods operating with known ground-truth camera are in the top part of the table, while the bottom part contains the methods that do not know the ground-truth camera.

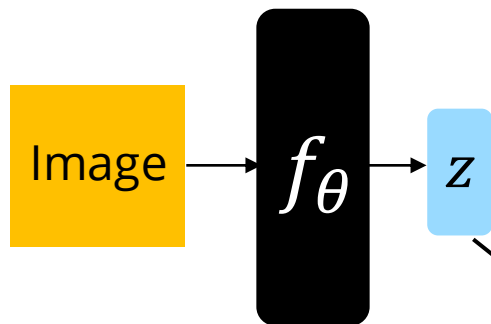
Methods	Acc. $\downarrow$	Comp. $\downarrow$	Overall $\downarrow$	Time
DUST3R	1.167	0.842	1.005	~ 7 s
MASt3R	0.968	0.684	0.826	~ 9 s
Ours (Point)	<u>0.901</u>	<u>0.518</u>	<u>0.709</u>	~ 0.2 s
Ours (Depth + Cam)	0.873	0.482	0.677	~ 0.2 s

Table 3. **Point Map Estimation on ETH3D [97].** DUST3R and MASt3R use global alignment while ours is feed-forward and hence, much faster. The row *Ours (Point)* indicates the results using the point map head directly, while *Ours (Depth + Cam)* denotes constructing point clouds from the depth map head combined with the camera head.

Feed-forward prediction: fast but less accurate

Two Families of 3D: Feed-forward vs Optimization

1. Feed-forward prediction



low quality

Scalable; data-driven
generalization

2. Optimization in the 3D space

Diagram illustrating the optimization process: A latent representation z (blue box) is used to minimize the reconstruction loss, expressed as $\min_z ||f_\theta(z) - \text{Image}||$. A curved arrow indicates that the feed-forward prediction z is used as initialization for the optimization, adding a prior loss $+ L_{prior}$.



Slow; no priors

Flexible; high fidelity

Use feed-forward as initialization

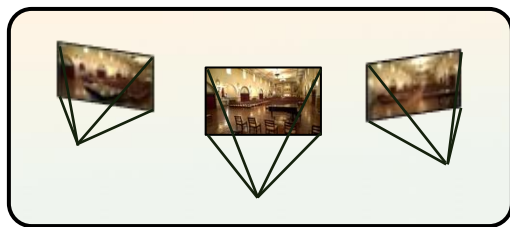
1. VGGT: Visual Geometry Grounded Transformer (Wang, J.; et al., CVPR 2025, Best Paper)

2. 3D Gaussian Splatting for Real-Time Radiance Field Rendering (Kerbl, B, et al, ACM TOG 2023, Best Paper)

Fast Refinement With Foundation Priors

$$\min_z ||g(z) - \text{Image}|| + L_{\text{prior}}$$

Optimization time: ~10 seconds for 3 views



Images

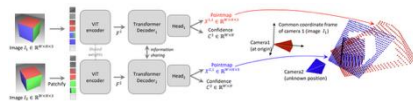
Initialization



Optimization

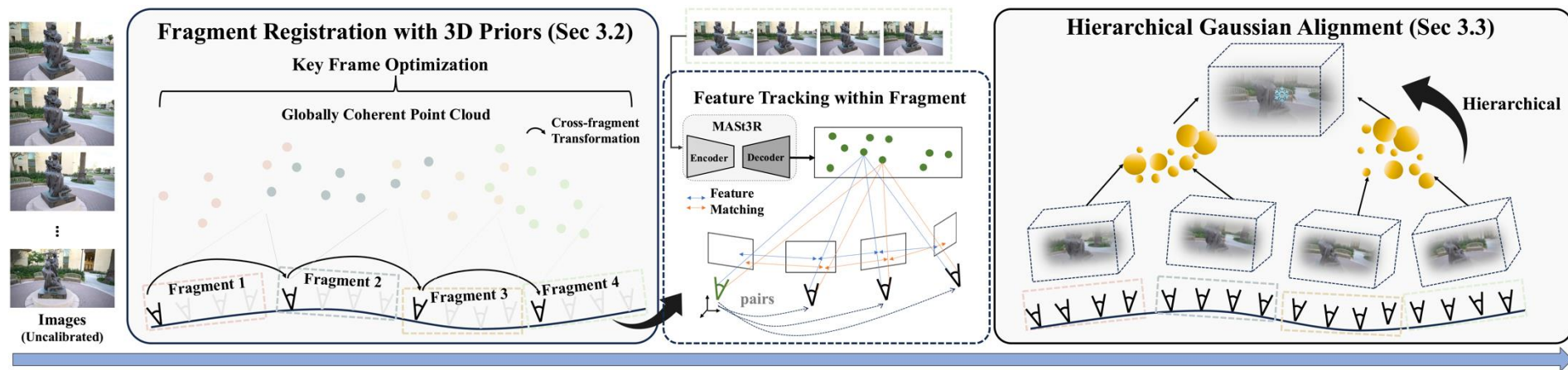


(T_v, K_v, G_n)



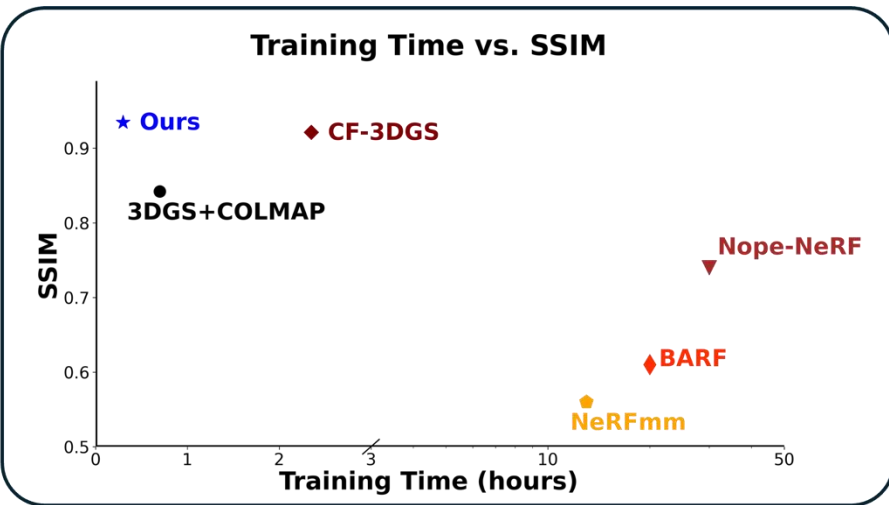
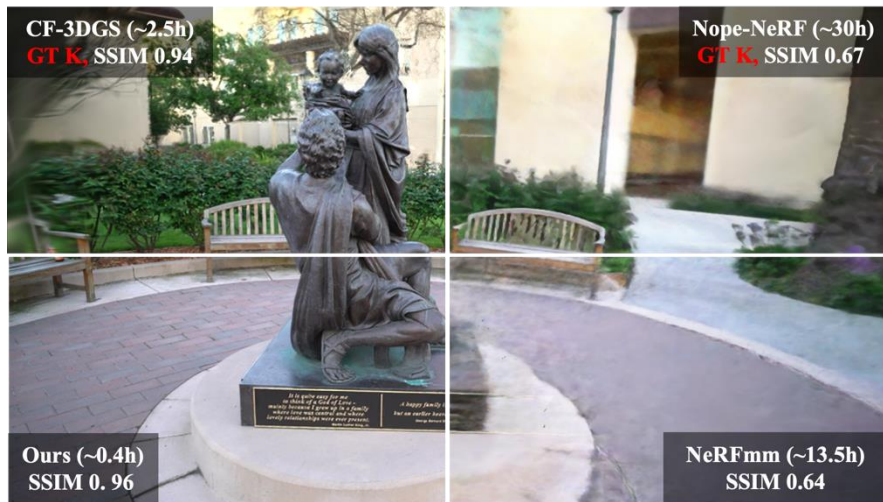
DUST3R, CVPR'2024

Video to 3D Reconstruction



Video -> Split to chunks -> 3D Reconstruction over chunks -> Alignment

Fast Refinement With Foundation Priors



Similar Principle Applies to SLAM

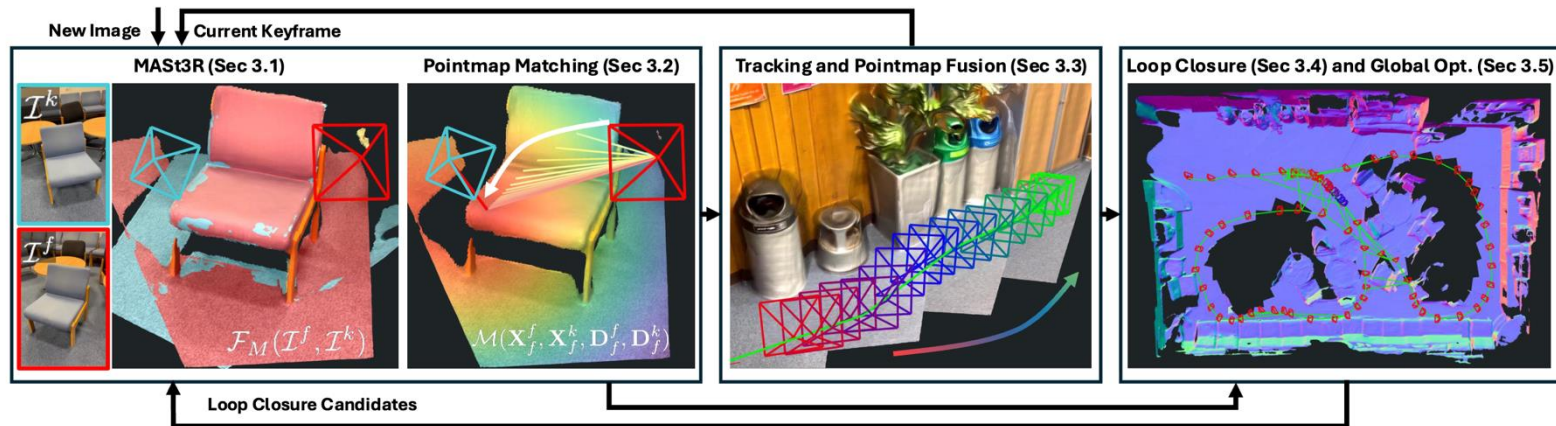


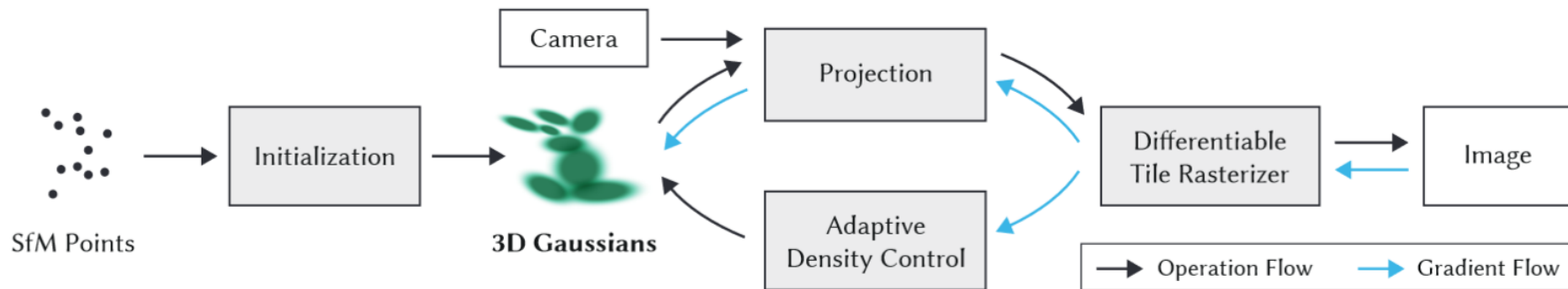
Figure 3. System diagram of MAST3R-SLAM. New images are tracked against the current keyframe by predicting a pointmap from MAST3R and finding pixel matches using our efficient iterative projection pointmap matching. Tracking estimates the current pose and performs local pointmap fusion. When new keyframes are added to the backend, loop closure candidates are selected by querying the retrieval database using encoded MAST3R features. Candidates are then decoded by MAST3R and if a sufficient number of matches is found, edges are added to the backend graph. Large-scale second-order optimisation achieves global consistency of poses and dense geometry.



Uncalibrated RGB
8x Playback



3DGS, the Heuristic Optimization



3DGS, the Performance

Dataset Method Metric	Mip-NeRF360						Tanks&Temples						Deep Blending					
	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1GB	0.719	21.08	0.379	25m5s	13.0	2.3GB	0.795	23.06	0.510	27m49s	11.2	2.7GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360	0.792 [↑]	27.69 [↑]	0.237 [↓]	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K	0.770	25.60	0.279	6m25s	160	523MB	0.767	21.20	0.280	6m55s	197	270MB	0.875	27.78	0.317	4m35s	172	386MB
Ours-30K	0.815	27.21	0.214	41m33s	134	734MB	0.841	23.14	0.183	26m54s	154	411MB	0.903	29.41	0.243	36m2s	137	676MB

Challenges with 3D-GS



Scene Bicycle



Model Size: 1400MB



Scene Garden



Model Size: 1400MB



Scene Stump



Model Size: 1100MB



Scene Truck

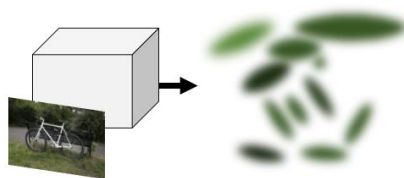


Model Size: 552MB

Significant amount of disk spaces are required in the explicit representation.

Poses significant scalability challenges and hindering the splatting efficiency.

Challenges with 3D-GS



Multi-view Images 3D Gaussians

Excessive Gaussian number



Often millions on each scene

Spherical Harmonic (SH) coefficient
requires high dimensions



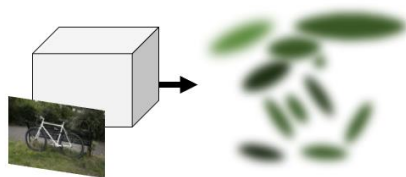
e.g., 3D-GS uses 3-degree SH, requires 48
dimension on each Gaussian

Inefficient storage



Similar attributes can be quantized
into lower bitwidth representation

Challenges with 3D-GS



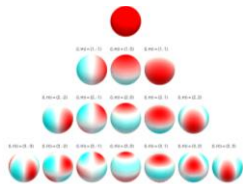
Multi-view Images 3D Gaussians

Excessive Gaussian number



Often million on each scene

Spherical Harmonic (SH) coefficient
requires high dimensions



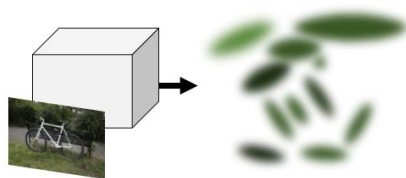
e.g., 3D-GS uses 3-degree SH, requires 48
dimension on each Gaussian

Inefficient storage



Similar attributes can be quantized
into lower bitwidth representation

Challenges with 3D-GS



Multi-view Images 3D Gaussians

Excessive Gaussian number



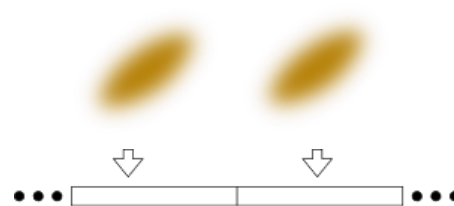
Often million on each scene

Spherical Harmonic (SH) coefficient requires high dimensions



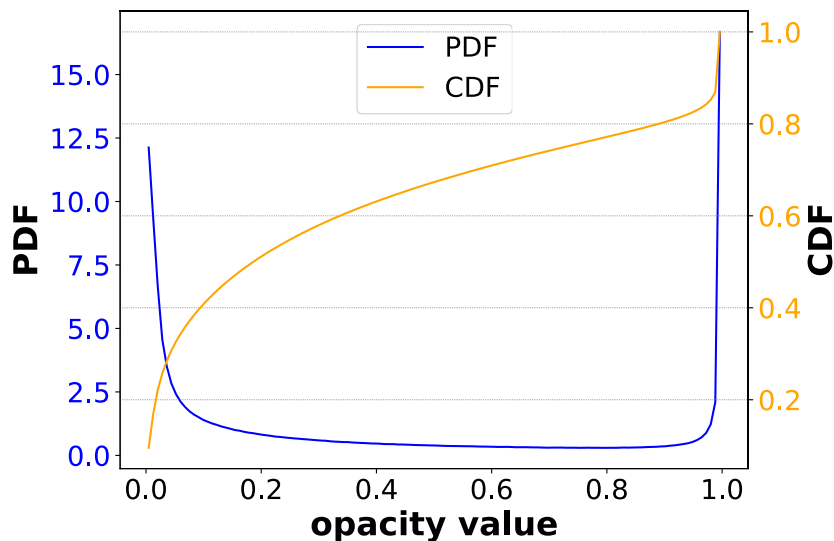
e.g., 3D-GS uses 3-degree SH, requires 48 dimension on each Gaussian

Inefficient storage



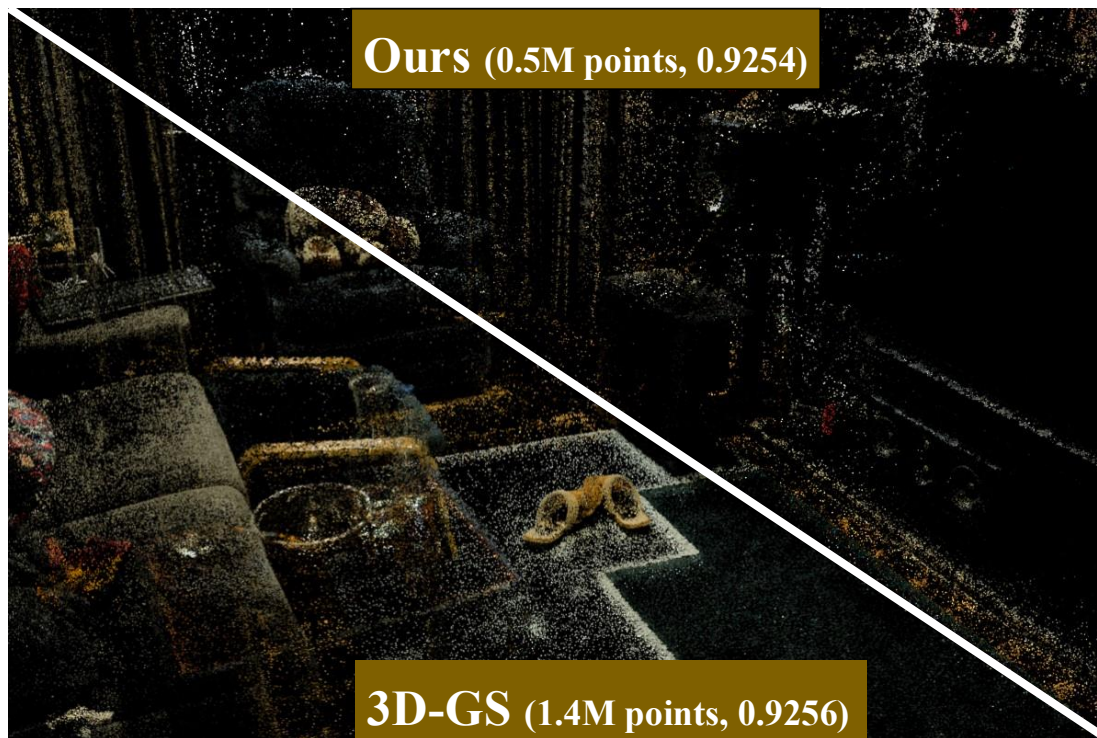
Similar attributes can be quantized into lower bitwidth representation

Excessive Gaussian Number



Simply using Gaussian Opacity as criteria results in significant performance drop, motivate us to find a better metric.

Excessive Gaussian Number



We propose to reduce the Gaussians via its global significance.

Excessive Gaussian Number



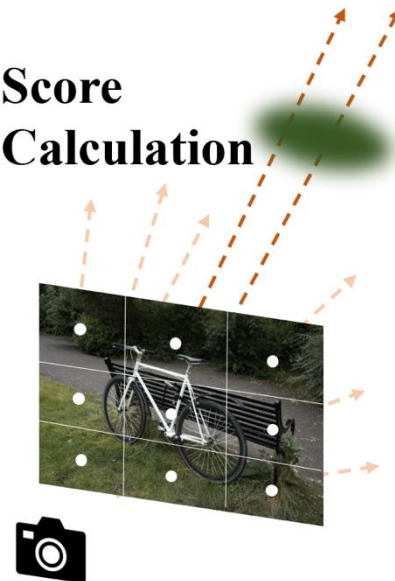
$$GS_j = \sum_{i=1}^{MHW} \mathbb{1}(G(\mathbf{X}_j), r_i) \cdot \sigma_j \cdot \gamma_j(\Sigma),$$



$$\gamma(\Sigma) = (V_{\text{norm}})^\beta,$$

$$V_{\text{norm}} = \min \left(\max \left(\frac{V(\Sigma)}{V_{\text{max90}}}, 0 \right), 1 \right).$$

**Score
Calculation**



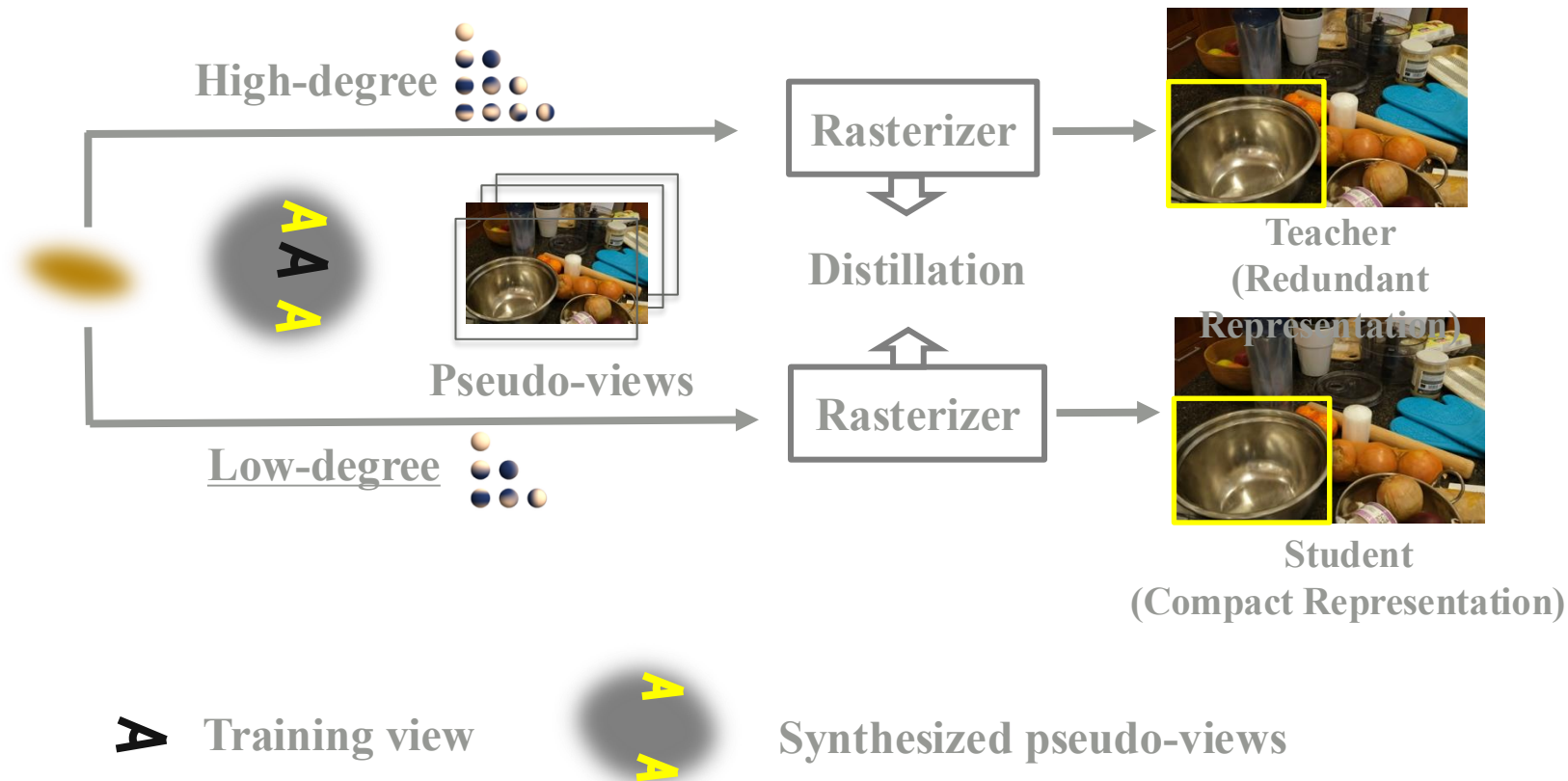
$G(\mathbf{X}_j)$: gaussian at location $\mathbf{X}_j$

$\mathbb{1}$: Hit indicator function

σ_j : capacity

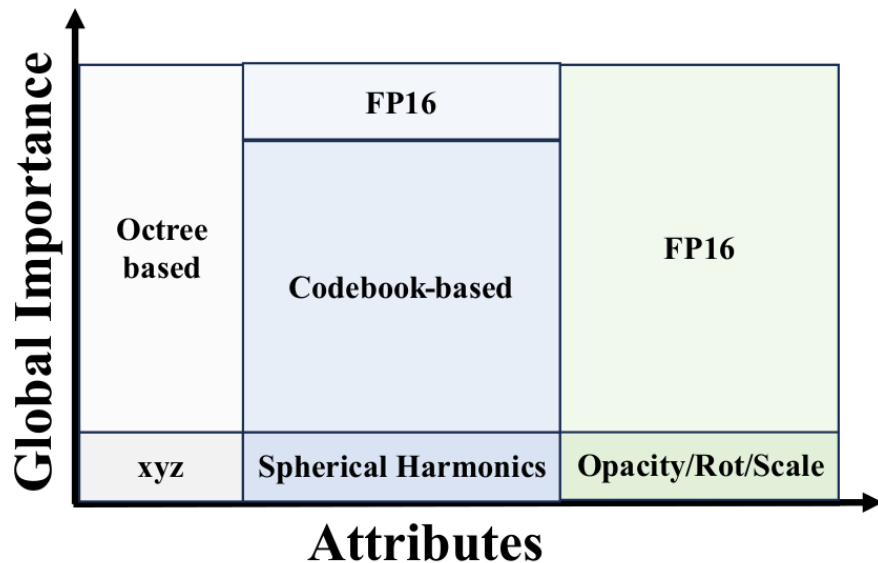
$\gamma(\Sigma)$: ne

Reduce the SH degree via Distillation



Vectree Quantization

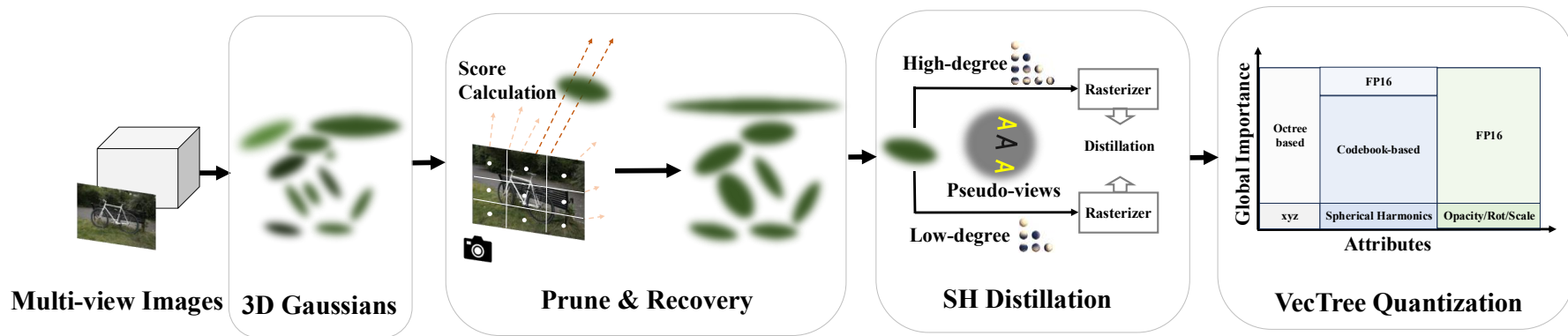
Color features can be represented by a compact codebook, while other attributes are sensitive for lossy compression.



Quantization and compression:

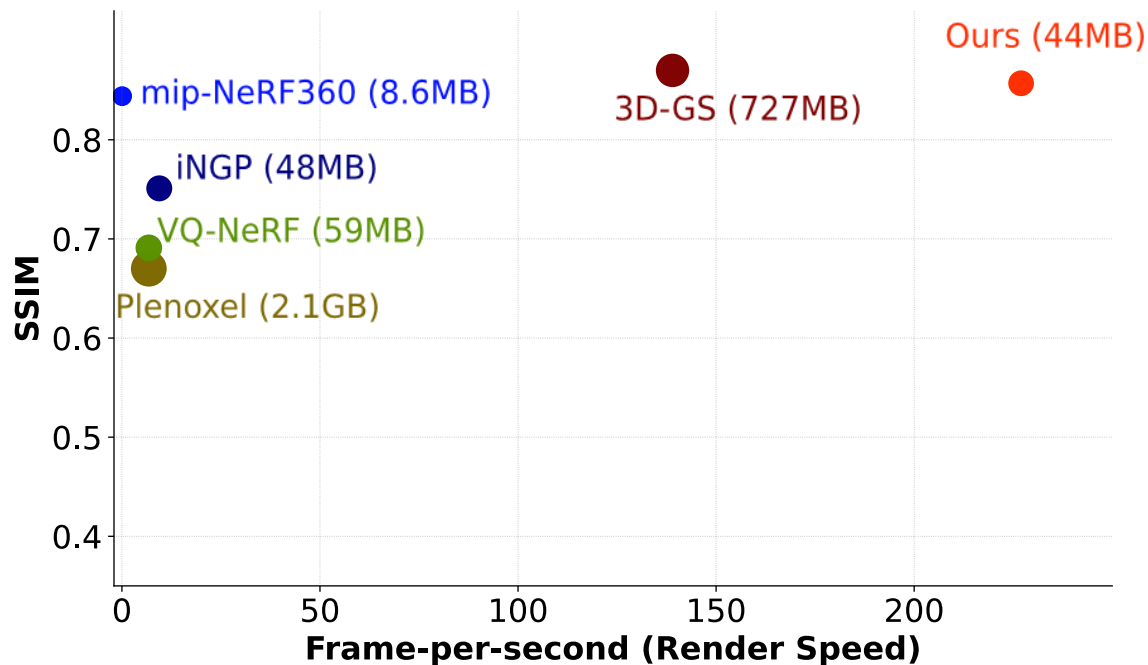
- **Octree Compression:**
 - Moderate compression ratio, and lossless.

LightGaussian, the Efficient Representation



- **Prune & Recovery:**
 - Reduce Gaussian number
- **SH Distillation:**
 - Distill SH coefficient into a more compact representation.
- **Vector Quantization:**
 - Hybrid compression different Gaussian attributes

Results



LightGaussian achieves 15x reduction in model size, and run at 200+FPS, thanks to the compact representation.

Results

Methods	Mip-NeRF 360 Datasets					Tank and Temple Datasets				
	FPS↑	Size↓	PSNR↑	SSIM↑	LPIPS↓	FPS↑	Size↓	PSNR↑	SSIM↑	LPIPS↓
Plenoxels [64]	6.79	2100MB	23.62	0.670	0.443	11.2	2700MB	21.07	0.719	0.379
INGP-Big [42]	9.43	48MB	26.75	0.751	0.299	2.79	48MB	21.92	0.744	0.305
Mip-NeRF 360 [5]	0.06	8.6MB	29.23	0.844	0.207	0.09	8.6MB	22.21	0.759	0.257
VQ-DVGO [36]	6.53	59.01MB	25.86	0.691	0.365	-	-	-	-	-
3D-GS [30]	139	727MB	29.13	0.870	0.185	106	380MB	23.11	0.822	0.219
Ours	215	44.36MB	28.45	0.857	0.210	209	23.53MB	22.83	0.807	0.242

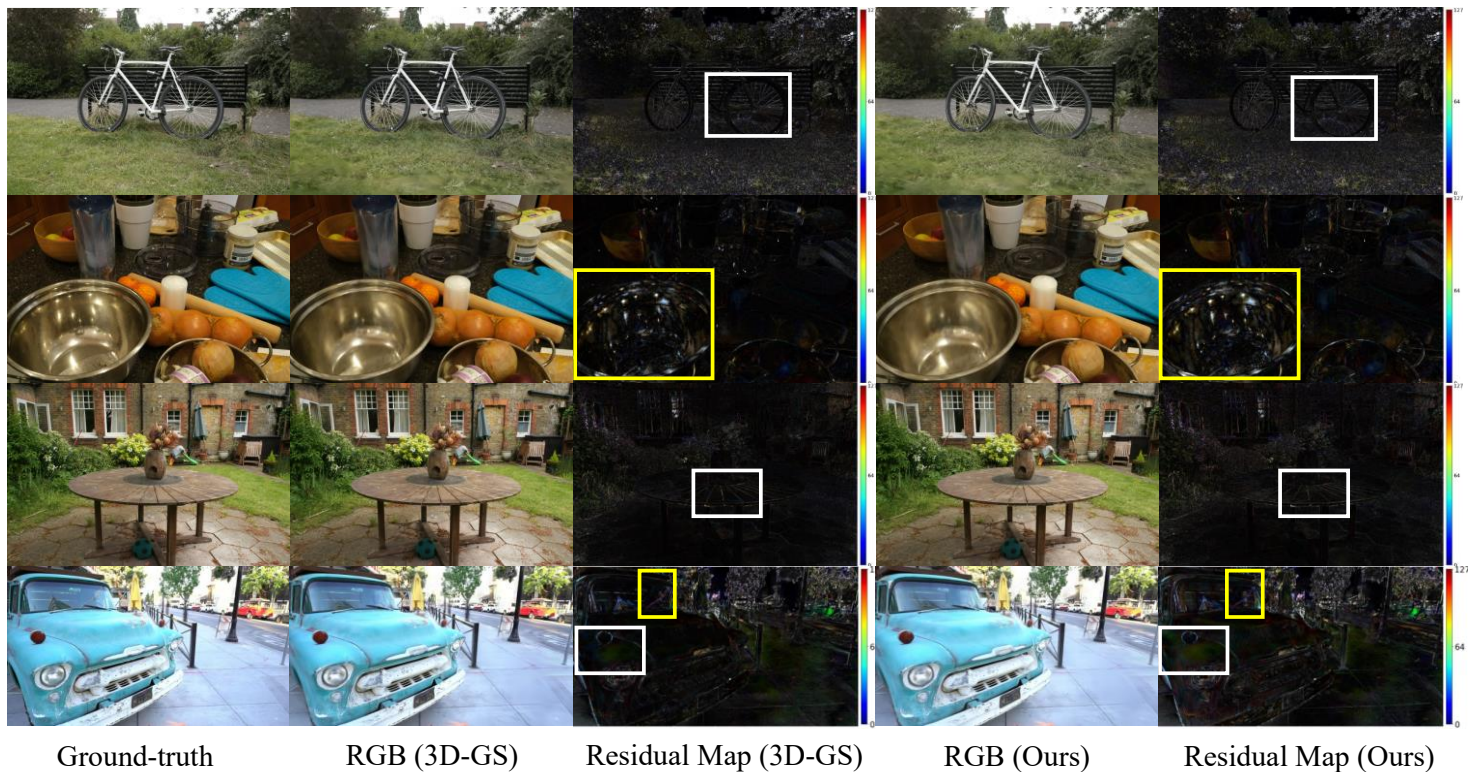
Quantitative Comparisons in Real-world Large-scale Scenes: LightGaussian (Ours) achieves the best balance between model size and accuracy and can run at a real-time speed.

Results

Exp#	Model	FPS $\uparrow$	Size $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
[1]	Baseline (3D-GS [30])	192.05	353MB	31.68	0.926	0.200
[2]	+ Gaussian Pruning.	312.30	116MB	30.32	0.911	0.222
[3]	+ Co-adaptation	303.99	116MB	31.85	0.925	0.206
[4]	+ SH Compactness.	318.97	77MB	30.54	0.914	0.217
[5]	+ Distillation	304.20	77MB	31.47	0.922	0.211
[6]	+ Pseudo-views	300.60	77MB	31.59	0.923	0.211
[7]	+ Codebook Quant.	300.60	23MB	31.26	0.917	0.220
[8]	+ Octree Compression.	300.60	20MB	31.26	0.917	0.220
[9]	LightGaussian (Ours)	300.60	20MB	31.26	0.917	0.220

- Gaussian Pruning[2] + Co-adaptation[3] reduces the Gaussian number, with trivial impact in quality.
- SH Compactness[4], along with distillation + pseudo-view augmentation preserve the specular reflection variance when changing viewpoint.
- VecTree Hybrid Compression [7][8] further compress the model with minimum quality decrease.

Results



We observe that LightGaussian preserves the specular reflections (yellow boxes) after converting to a compact format. Additionally, a slight lightness change is noted after the conversion (white boxes).